

1 Power On

You press the power button. Electricity is supplied to the motherboard.



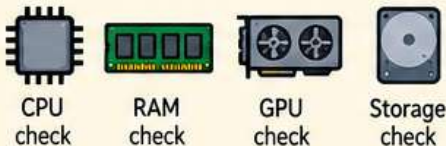
2 BIOS/UEFI Starts

The CPU starts executing the firmware (BIOS/UEFI) stored in a chip on the motherboard.



3 Hardware Initialization (POST)

BIOS/UEFI checks and initializes important hardware (CPU, RAM, GPU, storage, etc.). This is called Power On Self Test (POST).



4 Find Boot Device

BIOS/UEFI looks for a boot device based on the boot order (SSD/HDD/USB, etc.).

Boot Order

1. Windows Boot Manager (SSD)
2. USB Drive
3. SATA HDD
4. Network (PXE)

5 Load Bootloader

BIOS/UEFI loads the bootloader (from the selected boot device) into memory and gives control to it.

Bootloader

GNU GRUB

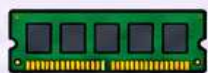
Ubuntu
Advanced options for Ubuntu
Memory test (memtest86+)

6 Bootloader Loads the Kernel

The bootloader loads the OS kernel (and initial RAM disk if needed) into RAM.



OS files
(on disk)



Kernel
(loaded into RAM)

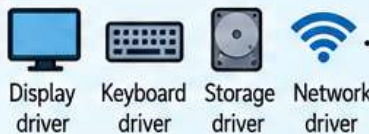
7 Kernel Starts

The kernel starts executing. It sets up low-level things like memory management, process management, and hardware control.

Loading Linux kernel ...
Initializing memory ...
Initializing drivers ...

8 Load Device Drivers

The kernel loads necessary drivers to control hardware (display, storage, keyboard, network, etc.).



9 Start System Services

The kernel starts system services daemons (e.g., udev, systemd) and initializes the system environment.

```
[ OK ] Started udev
[ OK ] Started display manager
[ OK ] Started network manager
[ OK ] Started system services
[ OK ] Reached multi-user target
```

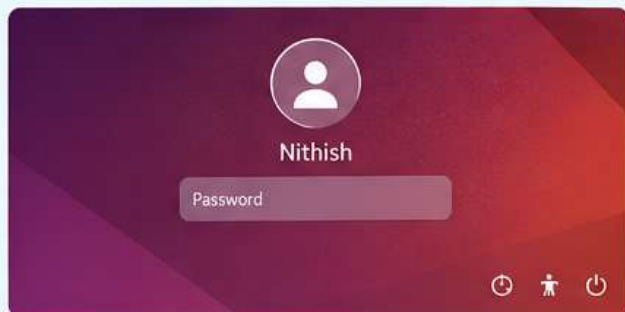
10 Start Display Server and Desktop Environment

The display server (e.g., X11/Wayland) starts and the desktop environment (GNOME, KDE, etc.) is loaded.



11 Show Login/Welcome Screen

Finally, the OS is ready and shows the login/welcome screen. You can now enter your password and start using the system.

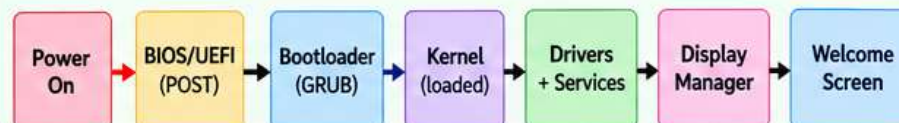


12 After Login (Desktop Loaded)

The desktop environment starts your user session, loads settings, and starts background applications. Now the OS is fully loaded and ready to use.



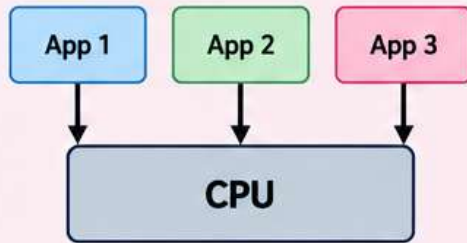
13 Summary Flow (Simplified)



Functions of an Operating System (Simplified)

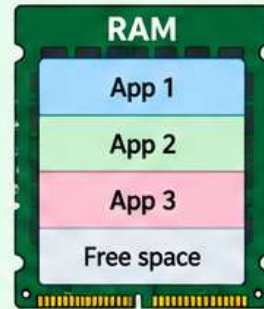
1 Process Management

- Runs programs (processes)
- Creates and terminates processes
- Shares CPU among processes (scheduling)



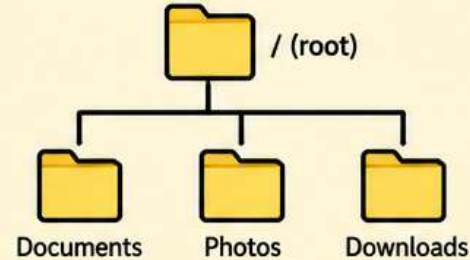
2 Memory Management

- Keeps track of used and free memory
- Allocates memory to programs
- Protects memory between processes



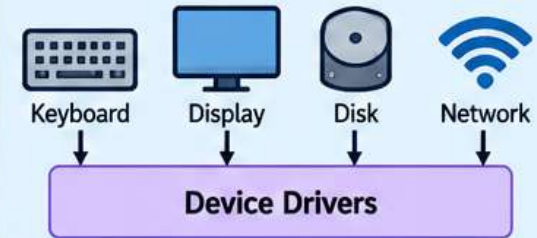
3 File System Management

- Stores and organizes files
- Creates, reads, writes, deletes files
- Manages folder structure
- Controls file permissions



4 Device Management

- Communicates with hardware through device drivers
- Controls devices (keyboard, mouse, display, disk, network, etc.)
- Handles input and output



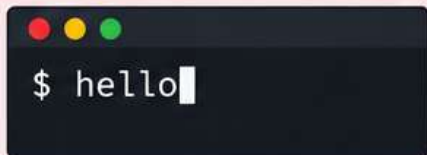
5 Security and Protection

- Controls who can access resources
- Protects files and memory
- Prevents one program from harming another

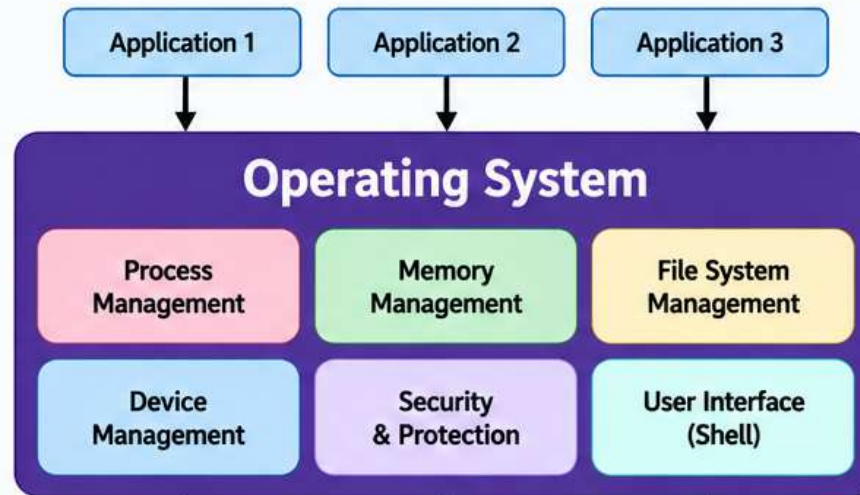


6 User Interface (Shell)

- Provides a way to interact with the OS
- Command-line interface (shell/terminal) or Graphical User Interface (GUI)

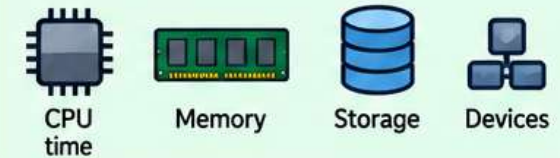


Terminal (Shell)



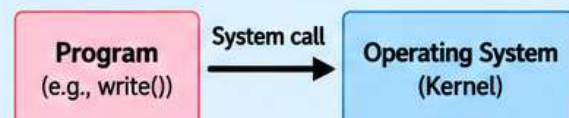
7 Resource Allocation

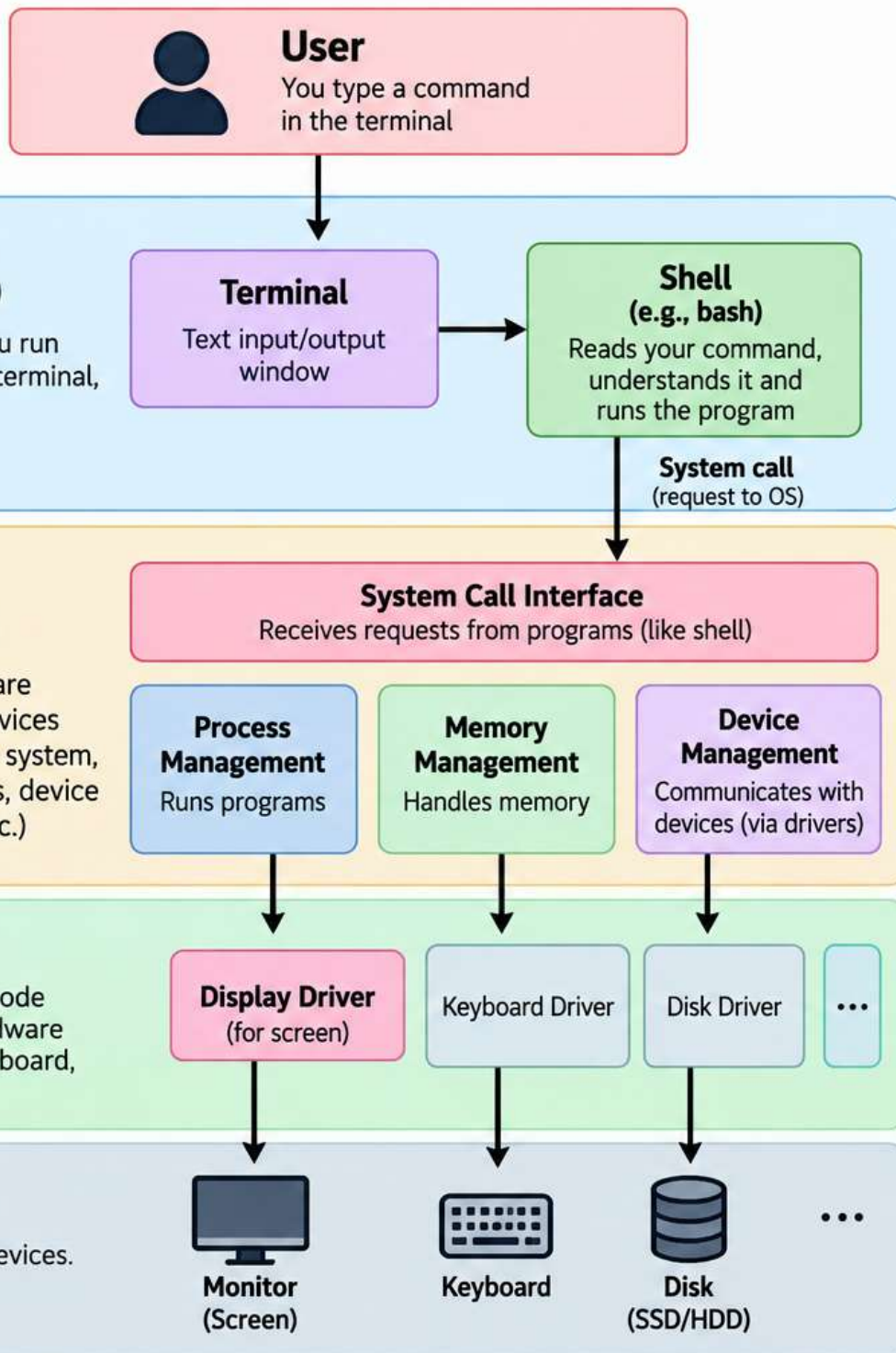
- Decides how to allocate CPU, memory, storage and devices to different programs
- Ensures fair and efficient use of resources



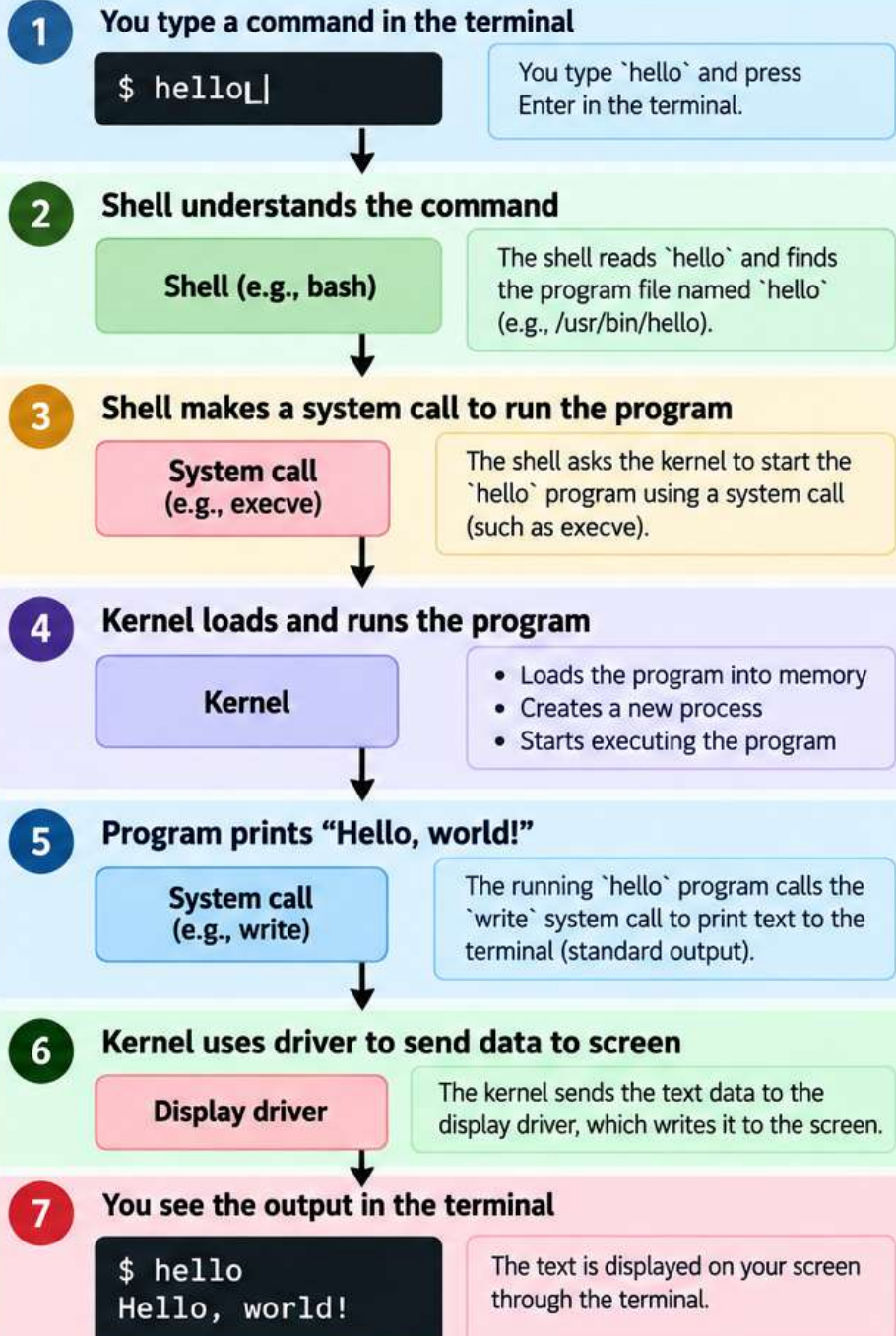
8 Provides Services to Programs

- Programs use system calls to request services from the OS (e.g., read file, print text, use network, etc.)
- The OS provides these services safely and efficiently.





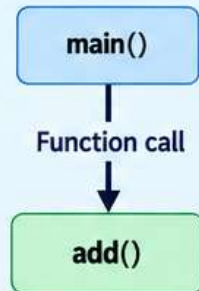
How `hello` Prints – Step by Step



1 Function call (inside a program)

A function call calls another function in your program or a library.

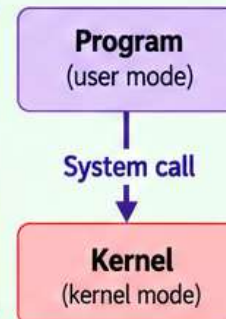
```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int result = add(10, 20); // function call  
}
```



2 System call (to the OS kernel)

A system call requests a service from the operating system (kernel).

```
#include <unistd.h>  
  
int main() {  
    write(1, "Hello\n", 6); // system call  
    return 0;  
}
```



3 What is an Operating System ?

The operating system (OS) is a software that manages hardware and provides services to programs.

Manages CPU
(processes)

Manages Memory

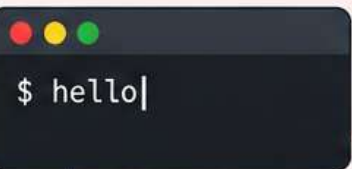
Manages Devices
(via drivers)

Provides services
(files, network, etc.)

4 How `hello` Prints – Step by Step (Full Flow)

1 You type the command in the terminal

You type `hello` and press Enter.



Terminal
(text input/output window)

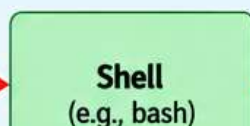


Types command

User

2 Shell runs the program

The shell (e.g., bash) reads your command, finds the program file `hello` and starts it.



3 Program calls a system call

The `hello` program calls `write()` to print the text.



4 Kernel handles the request

The kernel receives the system call, checks it and asks the appropriate device driver to write the text to the screen.



5 Display driver writes to the screen

The display driver sends the text to the screen (monitor).



6 You see the output

The text is displayed on your screen through the terminal.



Terminal
(output on screen)

User Space
(applications)

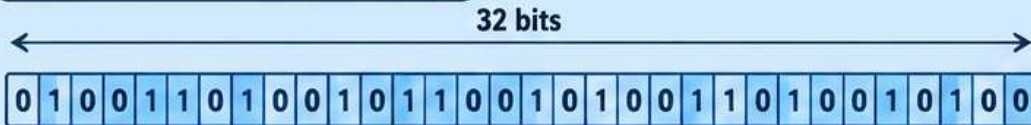
Kernel Space
(OS kernel)

Hardware
(devices like screen, keyboard, disk)

32-BIT OPERATING SYSTEM

A 32-bit OS is designed for a 32-bit CPU architecture, where the CPU uses 32-bit registers and 32-bit memory addresses.

1. CPU Register (32 bits)



A 32-bit register can hold 32 bits of data.

2. Memory Addressing (32-bit)

A 32-bit address can represent 2^{32} different memory addresses.

$2^{32} = 4,294,967,296$ addresses
 ≈ 4 GB (gigabytes) of address space

3. Memory Representation



A 32-bit OS can directly address up to 4 GB of memory (in theory; actual usable memory is often less).

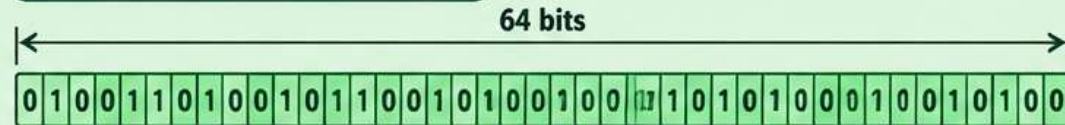
Key Points (32-bit)

- 32-bit registers
- 2^{32} memory addresses (≈ 4 GB)
- Limited RAM support (usually up to 4 GB)
- Older software and hardware
- Cannot run 64-bit applications

64-BIT OPERATING SYSTEM

A 64-bit OS is designed for a 64-bit CPU architecture, where the CPU uses 64-bit registers and 64-bit memory addresses.

1. CPU Register (64 bits)



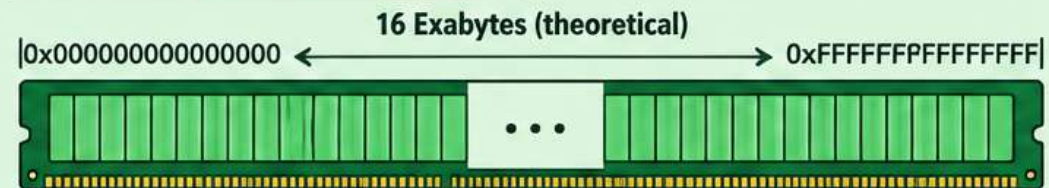
A 64-bit register can hold 64 bits of data.

2. Memory Addressing (64-bit)

A 64-bit address can represent 2^{64} different memory addresses.

$2^{64} = 18,446,744,073,709,551,616$ addresses
 ≈ 16 exabytes (EB) of address space
(much larger than current hardware needs)

3. Memory Representation



A 64-bit OS can address a vastly larger amount of memory (much more than current systems use).

Key Points (64-bit)

- 64-bit registers
- 2^{64} memory addresses (≈ 16 exabytes)
- Supports much larger RAM (8 GB, 16 GB, 32 GB, 128 GB, etc.)
- Modern operating systems and software
- Can run both 64-bit and (in most cases) 32-bit applications

1 CPU Register

A CPU register can hold a number.

32-bit register



32 bits = 4 bytes

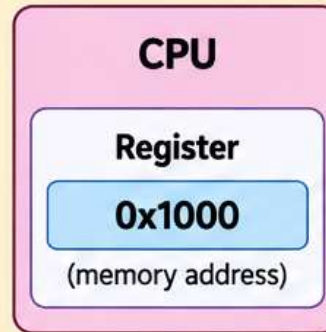
64-bit register



64 bits = 8 bytes

2 Register Holds Memory Address

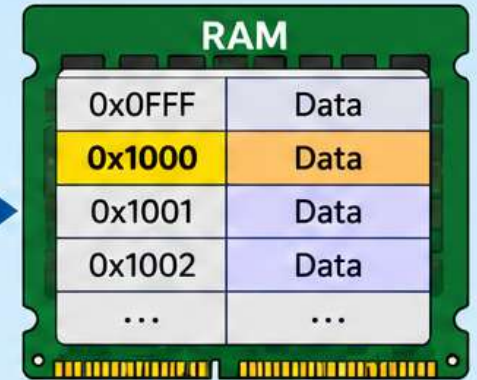
The register can store the address of a data location in RAM.



Use this address to read data

3 Accessing Data in RAM

The CPU uses the address to get the data from RAM.



4 Address Space (How many locations?)

With n address bits, the CPU can represent 2^n different addresses.

32-bit

$2^{32} = 4,294,967,296$ addresses

≈ 4 GB

address space
(in simple model)

64-bit

$2^{64} = 18,446,744,073,709,551,616$ addresses

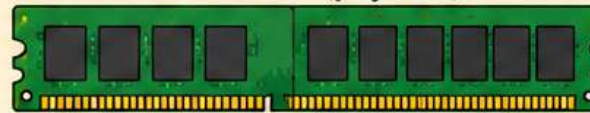
≈ 16 EB

address space
(theoretical)

5 Why 32-bit is a Problem

A 32-bit system can only address about 4 GB. If an application needs more memory, it cannot use all the available RAM directly.

16 GB RAM (physical)

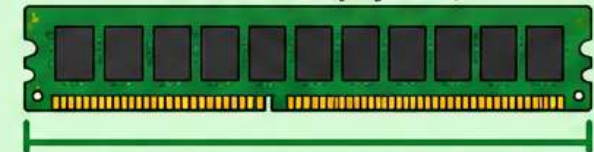


Only ~ 4 GB
can be used directly
(32-bit)

6 64-bit Solves This

A 64-bit system has a much larger address space, so applications can use much more of the available RAM.

16 GB RAM (physical)



Can use much more memory
(64-bit)

7 Summary

Wider registers
(64-bit)

Can represent
larger addresses

Larger address space

Can use much more
memory for applications

Better for modern
large applications
(e.g., video editing, games,
browsers, AI/ML, etc.)